# Hotel Booking System - Java Back End

Spring Boot 3 (Java 17) REST API with Spring Data JPA, bean validation and an in-memory H2 database. It checks room availability, prevents double bookings, calculates the price, issues a booking reference and supports cancellation. It is designed to work with the HTML front end.

### REST endpoints

```
GET    /api/rooms                                  list all rooms
GET    /api/rooms/available?checkIn=&checkOut=&guests=   rooms free for the dates
POST   /api/bookings                               create a booking (JSON body)
GET    /api/bookings/{reference}                   look up a booking
DELETE /api/bookings/{reference}                   cancel a booking
```

### Run it

```
mvn spring-boot:run
# API at http://localhost:8080   H2 console at /h2-console (JDBC URL jdbc:h2:mem:hoteldb)
```

### Example request

```
curl -X POST http://localhost:8080/api/bookings -H "Content-Type: application/json" -d '{
  "roomId": 2, "guestName": "Jane Doe", "guestEmail": "jane@example.com",
  "guestPhone": "5551234567", "checkIn": "2027-01-10", "checkOut": "2027-01-13", "guests": 2
}'
```

### Project structure

```
pom.xml
src/main/resources/application.properties
src/main/java/com/grandstay/hotel/HotelApplication.java
src/main/java/com/grandstay/hotel/model/Booking.java
src/main/java/com/grandstay/hotel/model/BookingStatus.java
src/main/java/com/grandstay/hotel/model/Room.java
src/main/java/com/grandstay/hotel/repository/BookingRepository.java
src/main/java/com/grandstay/hotel/repository/RoomRepository.java
src/main/java/com/grandstay/hotel/dto/BookingRequest.java
src/main/java/com/grandstay/hotel/exception/BookingException.java
src/main/java/com/grandstay/hotel/exception/GlobalExceptionHandler.java
src/main/java/com/grandstay/hotel/exception/ResourceNotFoundException.java
src/main/java/com/grandstay/hotel/service/BookingService.java
src/main/java/com/grandstay/hotel/controller/BookingController.java
src/main/java/com/grandstay/hotel/controller/RoomController.java
```

### pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.5</version>
  </parent>
  <groupId>com.grandstay</groupId>
  <artifactId>hotel</artifactId>
  <version>1.0.0</version>
  <properties><java.version>17</java.version></properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>com.h2database</groupId>
      <artifactId>h2</artifactId>
      <scope>runtime</scope>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

### resources/application.properties

```
server.port=8080
spring.datasource.url=jdbc:h2:mem:hoteldb
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
spring.h2.console.enabled=true
# For MySQL, replace the datasource lines above with:
# spring.datasource.url=jdbc:mysql://localhost:3306/hoteldb
# spring.datasource.username=root
# spring.datasource.password=yourpassword
```

### com/grandstay/hotel/HotelApplication.java

```
package com.grandstay.hotel;

import com.grandstay.hotel.model.Room;
import com.grandstay.hotel.repository.RoomRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;

import java.math.BigDecimal;

@SpringBootApplication
public class HotelApplication {

    public static void main(String[] args) {
        SpringApplication.run(HotelApplication.class, args);
    }

    /** Seeds sample rooms on first start. */
    @Bean
```

```
    CommandLineRunner seed(RoomRepository rooms) {
        return args -> {
            if (rooms.count() > 0) return;
            rooms.save(new Room("Standard Room", "Queen bed, Wi-Fi, TV", 2, new BigDecimal("80")));
            rooms.save(new Room("Deluxe Room", "King bed, city view", 3, new BigDecimal("120")));
            rooms.save(new Room("Family Suite", "2 bedrooms, kitchenette", 4, new BigDecimal("180")));
            rooms.save(new Room("Executive Suite", "Lounge, balcony, breakfast", 4,
                    new BigDecimal("260")));
        };
    }
}
```

## com/grandstay/hotel/model/Booking.java

```
package com.grandstay.hotel.model;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDate;

@Entity
@Table(name = "bookings")
public class Booking {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String reference;

    @ManyToOne(optional = false, fetch = FetchType.EAGER)
    private Room room;

    @Column(nullable = false)
    private String guestName;

    @Column(nullable = false)
    private String guestEmail;

    private String guestPhone;

    @Column(nullable = false)
    private LocalDate checkIn;

    @Column(nullable = false)
    private LocalDate checkOut;

    private int guests;

    @Column(precision = 10, scale = 2)
    private BigDecimal totalPrice;

    @Enumerated(EnumType.STRING)
    private BookingStatus status = BookingStatus.CONFIRMED;

    public Long getId() { return id; }
    public String getReference() { return reference; }
    public void setReference(String reference) { this.reference = reference; }
    public Room getRoom() { return room; }
    public void setRoom(Room room) { this.room = room; }
    public String getGuestName() { return guestName; }
    public void setGuestName(String guestName) { this.guestName = guestName; }
    public String getGuestEmail() { return guestEmail; }
    public void setGuestEmail(String guestEmail) { this.guestEmail = guestEmail; }
    public String getGuestPhone() { return guestPhone; }
    public void setGuestPhone(String guestPhone) { this.guestPhone = guestPhone; }
    public LocalDate getCheckIn() { return checkIn; }
    public void setCheckIn(LocalDate checkIn) { this.checkIn = checkIn; }
    public LocalDate getCheckOut() { return checkOut; }
    public void setCheckOut(LocalDate checkOut) { this.checkOut = checkOut; }
    public int getGuests() { return guests; }
    public void setGuests(int guests) { this.guests = guests; }
    public BigDecimal getTotalPrice() { return totalPrice; }
    public void setTotalPrice(BigDecimal totalPrice) { this.totalPrice = totalPrice; }
    public BookingStatus getStatus() { return status; }
    public void setStatus(BookingStatus status) { this.status = status; }
}
```

## com/grandstay/hotel/model/BookingStatus.java

```
package com.grandstay.hotel.model;
public enum BookingStatus {
```

```
    CONFIRMED,
    CANCELLED
}
```

### com/grandstay/hotel/model/Room.java

```
package com.grandstay.hotel.model;

import jakarta.persistence.*;
import java.math.BigDecimal;

@Entity
@Table(name = "rooms")
public class Room {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    private String description;

    @Column(nullable = false)
    private int capacity;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal pricePerNight;

    protected Room() { }

    public Room(String name, String description, int capacity, BigDecimal pricePerNight) {
        this.name = name;
        this.description = description;
        this.capacity = capacity;
        this.pricePerNight = pricePerNight;
    }

    public Long getId() { return id; }
    public String getName() { return name; }
    public String getDescription() { return description; }
    public int getCapacity() { return capacity; }
    public BigDecimal getPricePerNight() { return pricePerNight; }
}
```

### com/grandstay/hotel/repository/BookingRepository.java

```
package com.grandstay.hotel.repository;

import com.grandstay.hotel.model.Booking;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.time.LocalDate;
import java.util.Optional;

public interface BookingRepository extends JpaRepository<Booking, Long> {

    Optional<Booking> findByReference(String reference);

    @Query("""
           select count(b) from Booking b
           where b.room.id = :roomId
             and b.status = com.grandstay.hotel.model.BookingStatus.CONFIRMED
             and b.checkIn < :checkOut
             and b.checkOut > :checkIn
           """)
    long countOverlapping(@Param("roomId") Long roomId,
                          @Param("checkIn") LocalDate checkIn,
                          @Param("checkOut") LocalDate checkOut);
}
```

### com/grandstay/hotel/repository/RoomRepository.java

```
package com.grandstay.hotel.repository;

import com.grandstay.hotel.model.Room;
import jakarta.persistence.LockModeType;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Lock;
```

```
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.time.LocalDate;
import java.util.List;
import java.util.Optional;

public interface RoomRepository extends JpaRepository<Room, Long> {

    /** Rooms that fit the guests and have no confirmed booking overlapping the dates. */
    @Query("""
           select r from Room r
           where r.capacity >= :guests
             and r.id not in (
                 select b.room.id from Booking b
                 where b.status = com.grandstay.hotel.model.BookingStatus.CONFIRMED
                   and b.checkIn < :checkOut
                   and b.checkOut > :checkIn)
           order by r.pricePerNight
           """)
    List<Room> findAvailable(@Param("checkIn") LocalDate checkIn,
                             @Param("checkOut") LocalDate checkOut,
                             @Param("guests") int guests);

    /** Locks the room row so two people cannot book the same dates at once. */
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select r from Room r where r.id = :id")
    Optional<Room> findByIdForUpdate(@Param("id") Long id);
}
```

## com/grandstay/hotel/dto/BookingRequest.java

```
package com.grandstay.hotel.dto;

import jakarta.validation.constraints.*;
import java.time.LocalDate;

public record BookingRequest(
        @NotNull Long roomId,
        @NotBlank String guestName,
        @NotBlank @Email String guestEmail,
        @NotBlank @Size(min = 7, max = 20) String guestPhone,
        @NotNull @FutureOrPresent LocalDate checkIn,
        @NotNull @Future LocalDate checkOut,
        @Min(1) int guests) {
}
```

## com/grandstay/hotel/exception/BookingException.java

```
package com.grandstay.hotel.exception;

/** Thrown for business-rule violations (bad dates, room taken, too many guests). */
public class BookingException extends RuntimeException {
    public BookingException(String message) {
        super(message);
    }
}
```

## com/grandstay/hotel/exception/GlobalExceptionHandler.java

```
package com.grandstay.hotel.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.Instant;
import java.util.LinkedHashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<Map<String, Object>> notFound(ResourceNotFoundException ex) {
        return body(HttpStatus.NOT_FOUND, ex.getMessage());
    }

    @ExceptionHandler(BookingException.class)
    public ResponseEntity<Map<String, Object>> conflict(BookingException ex) {
```

```
        return body(HttpStatus.CONFLICT, ex.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> invalid(MethodArgumentNotValidException ex) {
        String msg = ex.getBindingResult().getFieldErrors().stream()
                .map(f -> f.getField() + ": " + f.getDefaultMessage())
                .reduce((a, b) -> a + "; " + b).orElse("Validation failed");
        return body(HttpStatus.BAD_REQUEST, msg);
    }

    private ResponseEntity<Map<String, Object>> body(HttpStatus status, String message) {
        Map<String, Object> map = new LinkedHashMap<>();
        map.put("timestamp", Instant.now().toString());
        map.put("status", status.value());
        map.put("error", message);
        return ResponseEntity.status(status).body(map);
    }
}
```

### com/grandstay/hotel/exception/ResourceNotFoundException.java

```
package com.grandstay.hotel.exception;

public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

### com/grandstay/hotel/service/BookingService.java

```
package com.grandstay.hotel.service;

import com.grandstay.hotel.dto.BookingRequest;
import com.grandstay.hotel.exception.BookingException;
import com.grandstay.hotel.exception.ResourceNotFoundException;
import com.grandstay.hotel.model.Booking;
import com.grandstay.hotel.model.BookingStatus;
import com.grandstay.hotel.model.Room;
import com.grandstay.hotel.repository.BookingRepository;
import com.grandstay.hotel.repository.RoomRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.security.SecureRandom;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.List;

@Service
public class BookingService {

    private static final SecureRandom RANDOM = new SecureRandom();

    private final RoomRepository rooms;
    private final BookingRepository bookings;

    public BookingService(RoomRepository rooms, BookingRepository bookings) {
        this.rooms = rooms;
        this.bookings = bookings;
    }

    public List<Room> allRooms() {
        return rooms.findAll();
    }

    public List<Room> availableRooms(LocalDate checkIn, LocalDate checkOut, int guests) {
        validateDates(checkIn, checkOut);
        return rooms.findAvailable(checkIn, checkOut, guests);
    }

    @Transactional
    public Booking createBooking(BookingRequest req) {
        validateDates(req.checkIn(), req.checkOut());

        // Lock the room row so concurrent requests for the same room are serialised
        Room room = rooms.findByIdForUpdate(req.roomId())
                .orElseThrow(() -> new ResourceNotFoundException("Room not found: " + req.roomId()));

        if (req.guests() > room.getCapacity()) {
            throw new BookingException("Room sleeps at most " + room.getCapacity() + " guests");
```

```
        }
        if (bookings.countOverlapping(room.getId(), req.checkIn(), req.checkOut()) > 0) {
            throw new BookingException("Room is not available for the selected dates");
        }

        long nights = ChronoUnit.DAYS.between(req.checkIn(), req.checkOut());

        Booking b = new Booking();
        b.setReference(newReference());
        b.setRoom(room);
        b.setGuestName(req.guestName().trim());
        b.setGuestEmail(req.guestEmail().trim());
        b.setGuestPhone(req.guestPhone().trim());
        b.setCheckIn(req.checkIn());
        b.setCheckOut(req.checkOut());
        b.setGuests(req.guests());
        b.setTotalPrice(room.getPricePerNight().multiply(BigDecimal.valueOf(nights)));
        b.setStatus(BookingStatus.CONFIRMED);
        return bookings.save(b);
    }

    public Booking getByReference(String reference) {
        return bookings.findByReference(reference)
                .orElseThrow(() -> new ResourceNotFoundException("Booking not found: " + reference));
    }

    @Transactional
    public Booking cancel(String reference) {
        Booking b = getByReference(reference);
        if (b.getStatus() == BookingStatus.CANCELLED) {
            throw new BookingException("Booking is already cancelled");
        }
        if (!b.getCheckIn().isAfter(LocalDate.now())) {
            throw new BookingException("Bookings can only be cancelled before check-in");
        }
        b.setStatus(BookingStatus.CANCELLED);
        return bookings.save(b);
    }

    private void validateDates(LocalDate checkIn, LocalDate checkOut) {
        if (!checkOut.isAfter(checkIn)) {
            throw new BookingException("Check-out must be after check-in");
        }
        if (checkIn.isBefore(LocalDate.now())) {
            throw new BookingException("Check-in cannot be in the past");
        }
    }

    private String newReference() {
        String ref;
        do {
            ref = "GS" + (100000 + RANDOM.nextInt(900000));
        } while (bookings.findByReference(ref).isPresent());
        return ref;
    }
}
```

## com/grandstay/hotel/controller/BookingController.java

```
package com.grandstay.hotel.controller;

import com.grandstay.hotel.dto.BookingRequest;
import com.grandstay.hotel.model.Booking;
import com.grandstay.hotel.service.BookingService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/bookings")
@CrossOrigin(origins = "*")   // restrict to your front-end origin in production
public class BookingController {

    private final BookingService service;

    public BookingController(BookingService service) {
        this.service = service;
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Booking create(@Valid @RequestBody BookingRequest request) {
        return service.createBooking(request);
```

```
    }

    @GetMapping("/{reference}")
    public Booking get(@PathVariable String reference) {
        return service.getByReference(reference);
    }

    @DeleteMapping("/{reference}")
    public Booking cancel(@PathVariable String reference) {
        return service.cancel(reference);
    }
}
```

### com/grandstay/hotel/controller/RoomController.java

```
package com.grandstay.hotel.controller;

import com.grandstay.hotel.model.Room;
import com.grandstay.hotel.service.BookingService;
import org.springframework.format.annotation.DateTimeFormat;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDate;
import java.util.List;

@RestController
@RequestMapping("/api/rooms")
@CrossOrigin(origins = "*")   // restrict to your front-end origin in production
public class RoomController {

    private final BookingService service;

    public RoomController(BookingService service) {
        this.service = service;
    }

    @GetMapping
    public List<Room> all() {
        return service.allRooms();
    }

    @GetMapping("/available")
    public List<Room> available(
            @RequestParam @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate checkIn,
            @RequestParam @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate checkOut,
            @RequestParam(defaultValue = "1") int guests) {
        return service.availableRooms(checkIn, checkOut, guests);
    }
}
```